



Dissertation on

“Adaptive Study Planning for Self-Directed Learners”

Submitted in partial fulfilment of the requirements for the award of degree of

Master of Technology in

Data Science and Artificial Intelligence

UE20CS971 Project Phase - 1

Submitted by:

Rohit Saji

PES2PGE24DS201

Under the guidance of
Prof. Ramesh Prakash Guledgudd
PES University

May – October 2026

M.TECH IN DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

Certificate

This is to certify that the dissertation entitled “Adaptive Study Planning for Self-Directed Learners” is a bonafide work carried out by **Rohit Saji (PES2PGE24DS201)** in partial fulfilment for the completion of Project Phase - 1 in the Program of Study – Master of Technology in Data Science and Artificial Intelligence under the rules and regulations of PES University, Bengaluru.

Prof. Ramesh Prakash Guledgudd
PES University

Declaration

I hereby declare that the Project Phase - 1 entitled “Adaptive Study Planning for Self-Directed Learners” has been carried out under the guidance of Prof. Ramesh Prakash Guledgudd, and submitted in partial fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science and Artificial Intelligence of PES University, Bengaluru, during the academic semester May – October 2026.

Acknowledgement

I would like to express my gratitude to my guide for continuous guidance and support throughout this project, and to the project coordinators for their help. I thank the Chairperson and faculty of the Department, PES University, for their support, and the university leadership for the opportunities provided. Finally, I extend heartfelt thanks to my family and friends for their constant encouragement.

Abstract

Self-directed learners routinely commit to a fixed study plan and fall behind or move ahead within days, yet the plan does not adapt and the learner has no reliable signal of how far off-track they are. This work develops an adaptive study-planning system for a single self-directed learner and evaluates the algorithms behind it. The system estimates a learner’s true study pace from their own session history using hierarchical Bayesian calibration with a shrinkage-regularized context model, detects behavioural shifts in that pace using CUSUM change-point detection, and projects the completion date with a calibrated uncertainty band using Gaussian-process regression, regenerating the plan when the learner’s realised pace diverges from the schedule. Each component is compared against established baselines on a literature-grounded synthetic dataset with known ground truth, using held-out evaluation with bootstrap confidence intervals and Holm/Benjamini–Hochberg correction. Phase I delivers and evaluates this open-loop system and reports the comparison; Phase II will close the loop with assessment-based verification of genuine learning. The selected models are realised in a working local-first web application.

Contents

1	Introduction	11
1.1	Problem Statement	11
1.2	Objectives	12
1.3	Scope of the Project	12
1.4	Organization of the Report	12
2	Literature Survey	13
2.1	Learner Pace Modelling and Bayesian Calibration	13
2.2	Behavioural-Change Detection	14
2.3	Progress Prediction with Gaussian Processes	15
2.4	Adaptive Scheduling and Study-Plan Generation	16
2.5	Self-Regulated-Learning Analytics	17
2.6	Methodology for Sourcing the Surveyed Models	17
2.7	Summary of Approaches	18
3	System Requirements Specification	19
3.1	Hardware Requirements	19
3.2	Software Requirements	19
3.3	Functional Requirements	20
3.4	Non-Functional Requirements	20
4	Proposed Methodology	22
4.1	The Adaptive Planning Loop	22
4.2	Candidate Models and Selected Approach	23
4.2.1	Pace Calibration	23
4.2.2	Change Detection	23
4.2.3	Progress Projection	23
4.2.4	Schedule Generation	24
4.3	Comparison Methodology	24
4.4	System Design and Evolution	25
4.5	System Architecture	27
5	Implementation Details	29
5.1	Development Environment	29
5.2	Datasets and Their Sources	29
5.3	Model Implementation	30
5.4	Evaluation Metrics and Reproducibility	31
5.5	The Phase-1 Application	31
5.5.1	Plan Creation: Onboarding and Roadmap Generation	32

5.5.2	Study Execution: the Session Runtime	32
5.5.3	Progress and Adaptation: Home, Week, and Replan	34
5.5.4	Plan Lifecycle: the Roadmap Calendar and Roadmaps Dashboard	35
5.5.5	The Local-First Data Architecture	35
6	Results and Discussion	38
6.1	Pace Calibration	38
6.2	Change Detection	39
6.3	Progress Projection	40
6.4	Schedule Generation	40
6.5	Discussion	41
7	Conclusion and Future Work	42
7.1	Conclusion	42
7.2	Future Work	43

List of Figures

2.1	Schematic illustration of Bayesian pace calibration by Equation (2.1): a broad prior over the pace ratio (dashed) tightens to a narrower posterior (solid) as session evidence accumulates. The values shown are illustrative, not measured.	14
2.2	Schematic illustration of one-sided CUSUM detection by Equation (2.2): the statistic stays near zero while the pace is stable and accumulates after a shift near $t = 10$, raising an alarm when it crosses the threshold h . The values shown are illustrative.	15
2.3	Schematic illustration of a Gaussian-process burn-up projection: observed cumulative progress (solid) is extrapolated to completion (dashed) with a 95% band that widens with the horizon; the dotted line marks 100% completion. The values shown are illustrative.	16
2.4	The predict-then-optimize architecture [14] this project adopts and extends: a predictive model estimates the learner's pace, and an optimizer turns the estimate into a schedule.	16
2.5	The self-regulated-learning analytics loop the reviewed work supports: trace data yields validated indicators that are fed back to the learner. The reviewed systems stop at feedback; this project extends the loop by feeding the signal into replanning.	17
4.1	The open adaptive loop. Logged sessions drive pace calibration, change detection, and progress projection; a detected shift raises a recalibration prompt, and the learner decides whether to regenerate the schedule, which in turn shapes future sessions.	22
4.2	The offline comparison pipeline. The synthetic generator supplies ground truth; each candidate is scored on per-component metrics; a bootstrap confidence interval and Holm/Benjamini–Hochberg correction gate the result before a model is selected. The oracle is an upper-bound reference, not a deployable method.	25
4.3	Initial design (predict-then-optimize). The intelligence service runs all four components, and the schedule generator packs each material onto a fixed day, returning a dated <code>Slot</code> grid; a replan re-runs the packer on the service. 26	
4.4	Revised design (capacity-based booking), the delivered system. Progress projection and the booking engine run client-side; Bayesian and enriched-shrinkage calibration and CUSUM detection run in the Python service; Supabase is the data hub; the offline research pipeline supplies the selected models. The closed-loop feedback from verified mastery is drawn dashed as Phase-II work.	27

5.1	Onboarding: the generated booking plan produced from the learner’s deadline, weekly capacity, and materials.	33
5.2	A study session in progress, with the timer and material chosen at the start of the session.	33
5.3	The Home dashboard: up-next booking, streak, calibrated pace, and projected finish date.	34
5.4	The Week view: the week’s verdict, the daily-minutes chart, and the burn-up projection.	35
5.5	The Replan screen: three levers to adjust the plan when the projection diverges from the deadline.	36
5.6	The Roadmap calendar: the plan shown as a month grid of bookings. . .	36
5.7	The Roadmaps dashboard: the active plan, drafts, and history over their lifecycle.	37
6.1	Interval coverage against the nominal 0.95 (dashed) by history-length band, frozen regime. The plain Gaussian-process interval under-covers badly; the split-conformal correction restores near-nominal coverage.	40

List of Tables

2.1	Summary of Pillar-A approach families, with strengths and limitations for this project.	18
3.1	Hardware environment.	19
3.2	Software environment by tier.	20
4.1	Candidate models per component and the model selected for the delivered Phase-I system.	24
4.2	Evaluation metrics by component.	25
5.1	Development environment: tools and libraries.	29
5.2	Datasets used in Phase I and their sources.	30
5.3	Main packages and their roles.	31
6.1	Pace calibration: mean recovery error (lower is better) by history-length band, frozen regime. The oracle is the upper-bound reference.	38
6.2	Enriched-shrinkage minus hierarchical-Bayesian on the prospective context-prediction error (negative favours enriched-shrinkage), with 95% bootstrap intervals.	39
6.3	Change detection: mean detection latency (sessions, lower is better) and false-alarm rate by shift type, frozen regime. ADWIN did not fire on this data.	39
6.4	Progress projection: interval coverage (nominal 0.95) and point error in days, by band. Point error is shared between the plain and conformal intervals.	40
6.5	Schedule generation: mean absolute deadline drift (days) by material mix, reality-matched regime. Capacity violations were zero for the selected allocator.	41

Chapter 1

Introduction

This chapter sets out the problem the project addresses and the shape of the work that follows. Section 1.1 states the problem faced by self-directed learners and the motivation for an adaptive study planner. Section 1.2 lists the objectives of Phase I. Section 1.3 defines the scope of the work across the two project phases. Section 1.4 describes the organization of the remainder of the report.

1.1 Problem Statement

A self-directed learner is a person who works through online courses, examination preparation, or self-study material without an instructor setting the pace. Such a learner typically begins with a fixed plan, of the form “two hours a day, finish by this date.” The plan is a one-time estimate. It assumes that the learner will study at the rate they intended and that the material will take as long as they guessed.

Within a few days the plan no longer matches reality. The learner falls behind on some days and moves ahead on others, the estimate of how long each item takes proves wrong, and the fixed plan does not adjust. The learner is left without a trustworthy signal of how far off-track they are or when they will actually finish. The estimate that mattered at the start, the finish date, silently becomes inaccurate while the plan continues to display it as though it were still valid.

Existing tools address only one side of this problem. Time-tracking applications record how long a learner studied but do not plan; static plan generators produce a schedule once and never revise it against the rate the learner actually achieves. Neither learns the learner’s realised pace, and neither revises the finish-date estimate as evidence accumulates. The gap this project addresses is the absence of a planner that treats the learner’s own logged behaviour as evidence, updates its estimate of their pace as that evidence arrives, and regenerates the plan accordingly.

The difficulty is that the evidence is sparse. A single learner produces only a handful of study sessions before the plan first needs to adapt, so a method that requires large amounts of per-learner data is not usable here. The pace is also not stationary: it shifts when the learner’s circumstances change, and a useful system must distinguish a genuine shift from ordinary day-to-day variation before it acts. These two constraints, sparse data and non-stationary behaviour, shape the choice of methods in the chapters that follow.

1.2 Objectives

Phase I of the project pursues the following objectives.

1. Estimate a learner’s true study pace from their own session history using hierarchical Bayesian calibration with a context-aware shrinkage model, so that a usable estimate is available from few sessions.
2. Detect behavioural shifts in the learner’s pace using statistical change-point detection, so that the plan adapts to a genuine change rather than to noise.
3. Project the learner’s completion date with a calibrated uncertainty band, rather than a single point, using Gaussian-process regression over the accumulated progress.
4. Regenerate the study schedule from the learner’s realised capacity when the projected finish date diverges from the deadline.
5. Evaluate each component against established baselines on a synthetic dataset with known ground truth, using held-out evaluation with bootstrap confidence intervals and multiple-comparison correction.
6. Realise the selected models in a working, local-first web application for a single self-directed learner.

1.3 Scope of the Project

The work is delivered across two phases. **Phase I**, reported here, builds and evaluates the open-loop system. Its components are pace calibration, change detection, completion projection, and capacity-based schedule regeneration. Each is compared offline against baselines on synthetic data with known ground truth, and the selected models are realised in the application. The target user is the individual self-directed learner.

Phase II will close the loop and add assessment-based verification of genuine learning, so that the pace signal driving the plan is checked against evidence that studying produced learning rather than against self-reported time alone. Phase II will also validate the system longitudinally on a single learner’s real logged sessions. Multi-user, institutional, and instructor-mediated settings are out of scope for both phases.

1.4 Organization of the Report

The remainder of the report is organized as follows. Chapter 2 surveys the research the project draws on, grouped by technique, and states the gaps it addresses. Chapter 3 specifies the hardware, software, and functional requirements of the system. Chapter 4 describes the proposed methodology: the candidate models for each component, the offline comparison methodology, the system design and how it evolved during development, and the resulting architecture. Chapter 5 gives the implementation details, including the datasets and their sources and the realised application. Chapter 6 presents and discusses the comparison results. Chapter 7 concludes Phase I and sets out the work planned for Phase II.

Chapter 2

Literature Survey

This chapter surveys the techniques the project draws on, organized by the components of the adaptive planner and the analytics that motivate them. Section 2.1 reviews learner pace modelling and Bayesian calibration. Section 2.2 covers behavioural-change detection. Section 2.3 surveys progress prediction with Gaussian processes. Section 2.4 reviews adaptive scheduling and study-plan generation. Section 2.5 covers self-regulated-learning analytics. Section 2.6 describes the methodology used to source and screen the surveyed literature. Section 2.7 summarizes the approaches, states their limitations for this project, and positions the proposed solution.

2.1 Learner Pace Modelling and Bayesian Calibration

Estimating how fast a learner truly works, from limited per-learner data, is the foundation of the planner. Chen et al. [1] present a hierarchical Bayesian model of adaptive teaching and validate it through two online behavioural experiments with 312 adults, showing that hierarchical priors capture learner-to-learner variability without overfitting to sparse data. This is the statistical justification for estimating a single learner’s pace from few sessions. The model targets a teacher’s choice of examples rather than the planned-versus-actual pace this project estimates, so it supplies the prior structure but not the quantity to be calibrated.

The pace signal itself is simple to state. For a session the pace ratio is $r_t = a_t/p_t$, the ratio of active minutes a_t to planned minutes p_t , modelled as $r_t \sim \mathcal{N}(\mu, \sigma^2)$ with a prior $\mu \sim \mathcal{N}(\mu_0, \tau_0^2)$ centred at $\mu_0 = 1$ (the learner works at the planned rate). With n observed sessions the conjugate posterior is Gaussian,

$$p(\mu \mid r_{1:n}) = \mathcal{N}(\mu_n, \tau_n^2), \quad \tau_n^2 = \left(\frac{1}{\tau_0^2} + \frac{n}{\sigma^2} \right)^{-1}, \quad \mu_n = \tau_n^2 \left(\frac{\mu_0}{\tau_0^2} + \frac{1}{\sigma^2} \sum_{t=1}^n r_t \right). \quad (2.1)$$

When n is small the posterior stays close to the prior, which is what lets a usable estimate exist from a handful of sessions; as n grows the data term dominates and the estimate tightens around the learner’s realised rate, as illustrated in Figure 2.1.

Zanellati et al. [2] review hybrid knowledge-tracing models in a taxonomy of approaches that combine Bayesian posteriors with machine learning, and conclude that hybrid models consistently outperform either pure Bayesian Knowledge Tracing or pure deep learning. Zambrano et al. [3] analyse algorithmic bias in Bayesian Knowledge Tracing at scale and report that performance is essentially equal across demographic groups,

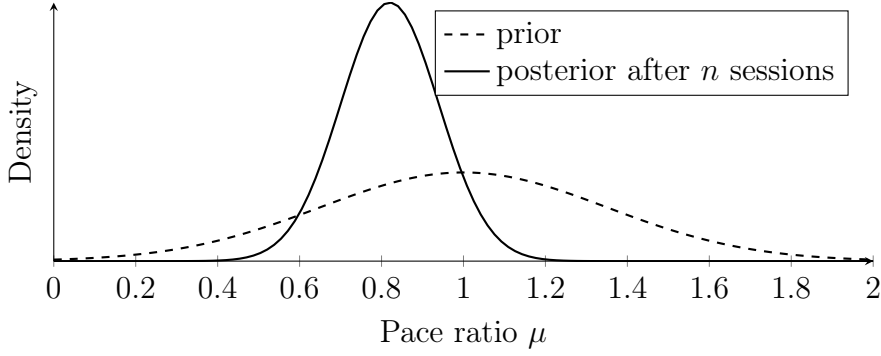


Figure 2.1: Schematic illustration of Bayesian pace calibration by Equation (2.1): a broad prior over the pace ratio (dashed) tightens to a narrower posterior (solid) as session evidence accumulates. The values shown are illustrative, not measured.

which supports the reliable use of Bayesian-family models across diverse learners. Mai et al. [4] couple deep temporal modelling with a probabilistic layer in a Transformer–Bayesian hybrid that recovers interpretable, calibrated knowledge estimates, reinforcing that a hybrid Bayesian design keeps transparency without sacrificing predictive accuracy. These works establish Bayesian calibration as sound for learner modelling, but they address correctness prediction rather than the planned-versus-actual pace estimation this project requires.

2.2 Behavioural-Change Detection

A learner’s pace is not stationary, and detecting when it shifts is what makes timely re-planning possible. Saqr et al. [5] apply critical-slowness indicators in an idiographic analysis of 1.67 million practice attempts from 9,401 students and find that 88.2% of disengaging learners exhibit detectable warning signals beforehand, which is strong evidence that per-learner behavioural shifts are recoverable from trace data. Qiao et al. [6] provide a PRISMA systematic review of statistical-process-control methods and document the rationale for CUSUM and EWMA control charts and their machine-learning-enhanced variants, though the reviewed applications target manufacturing rather than education.

The cumulative-sum chart is the detector this project builds on. A one-sided chart tracks the running deviation of the pace signal x_t from its in-control mean μ_0 ,

$$S_0 = 0, \quad S_t = \max(0, S_{t-1} + (x_t - \mu_0) - k), \quad \text{alarm if } S_t > h, \quad (2.2)$$

where the slack k absorbs ordinary day-to-day variation and h is the decision threshold [6]. The statistic stays near zero while the pace is stable and accumulates once a genuine shift occurs, raising an alarm when it crosses h , as illustrated in Figure 2.2.

Boca de Giuli et al. [7] combine control charts with continual learning to separate endogenous system shifts from one-time exogenous disruptions, a distinction that maps directly onto the need to recalibrate on genuine pace change while ignoring a one-off disruption. Cheng et al. [8] model behavioural change directly from heterogeneous student records for early at-risk prediction, deliberately retaining qualitative information that pure feature extraction discards. The limitation shared across these methods is that they detect a shift but prescribe no corrective action, which is the step this project adds by feeding the detected shift into replanning.

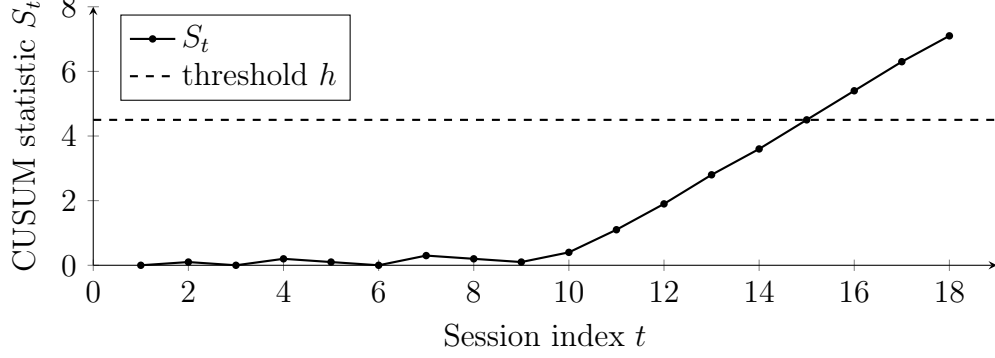


Figure 2.2: Schematic illustration of one-sided CUSUM detection by Equation (2.2): the statistic stays near zero while the pace is stable and accumulates after a shift near $t = 10$, raising an alarm when it crosses the threshold h . The values shown are illustrative.

2.3 Progress Prediction with Gaussian Processes

Projecting a completion date with a calibrated uncertainty band, rather than a single number, is what makes a projection trustworthy. Pérez-Suay et al. [9] apply Gaussian-process (GP) regression with an automatic-relevance-determination kernel to learning-management-system logs and achieve a correlation of $R = 0.89$ for predicting student marks, with interpretable feature weighting. Chen and Yao [10] combine black-hole optimisation for feature selection with a weighted GP-regression ensemble, reporting a root-mean-square error of 0.95 and a mean absolute error of 0.81, and show that GP predictions degrade gracefully as data becomes scarce.

A Gaussian process places a prior over progress curves and conditions it on the observed points. For a test time x_* the predictive distribution is Gaussian, with mean and variance

$$\bar{f}_* = \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{y}, \quad \mathbb{V}[f_*] = k(x_*, x_*) - \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{k}_*, \quad (2.3)$$

under a squared-exponential kernel with signal variance σ_f^2 and length scale ℓ ,

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{(x-x')^2}{2\ell^2}\right). \quad (2.4)$$

The predictive variance in Equation (2.3) is what supplies a calibrated band around the projected finish date, rather than a single point, as illustrated in Figure 2.3.

Lin et al. [11] introduce a latent Kronecker structure to scale GPs for learning-curve prediction, matching Transformer performance while retaining calibrated uncertainty. Beyond regression, Huang and Wang [12] use Gaussian-process classification with a meta-heuristic hyper-parameter search to predict student performance bands, and Kayande and Kukreja [13] report an integrated multi-modal machine-learning framework for engagement and dropout prediction on educational data. These works validate GP methods on educational data, but they apply the model cross-sectionally to a final outcome rather than to the longitudinal burn-up projection, with a running finish-date estimate, that this project needs.

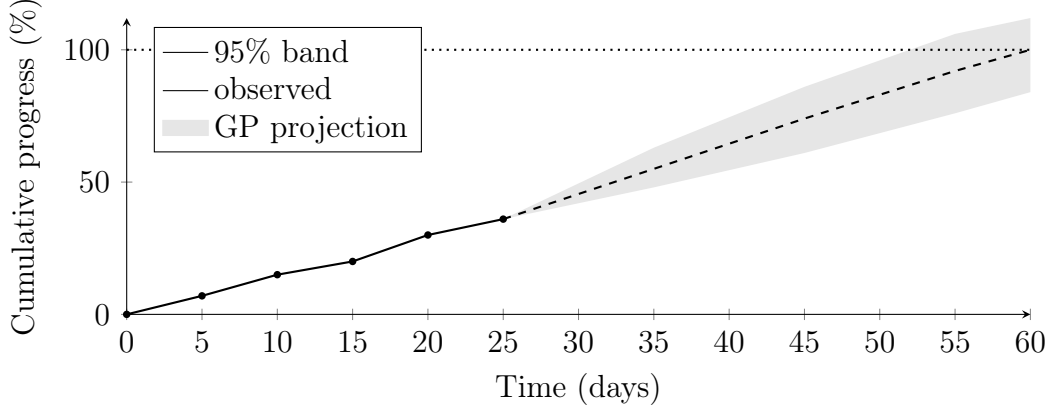


Figure 2.3: Schematic illustration of a Gaussian-process burn-up projection: observed cumulative progress (solid) is extrapolated to completion (dashed) with a 95% band that widens with the horizon; the dotted line marks 100% completion. The values shown are illustrative.

2.4 Adaptive Scheduling and Study-Plan Generation

The system must turn its estimates into a concrete, regenerated schedule. Islam et al. [14] couple an artificial neural network for grade prediction, at an accuracy of about 0.73, with dynamic programming for optimal schedule allocation, establishing the predict-then-optimize architecture this project takes as its Pillar-A base paper. The two stages are shown in Figure 2.4: a predictive model estimates the learner’s pace, and an optimizer turns that estimate into a schedule.

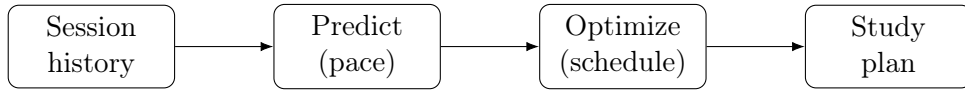


Figure 2.4: The predict-then-optimize architecture [14] this project adopts and extends: a predictive model estimates the learner’s pace, and an optimizer turns the estimate into a schedule.

The optimize step solves a scheduling problem by dynamic programming. Writing $V(t, \rho)$ for the best achievable cost from day t onward with remaining work ρ , the schedule follows a recurrence of the form

$$V(t, \rho) = \min_{0 \leq a \leq c_t} [\ell(a) + V(t + 1, \rho - a)], \quad (2.5)$$

where a is the study time allocated on day t , c_t is that day’s capacity, and ℓ penalises deadline drift [14]. Fabregar and Amorado [15] generate expert-quality study plans in 0.32 seconds using a greedy algorithm with prerequisite-constraint satisfaction, which demonstrates fast constraint-based, prerequisite-aware scheduling. Pagano et al. [16] report a controlled study of 2,064 students in which adaptive learning paths produce substantial efficiency gains over static ones, giving both large-scale evidence that adaptation helps and a template for an adaptive-versus-static evaluation design. Liu et al. [17] integrate a domain knowledge graph with AI-driven sequencing to construct personalised learning paths that respect concept prerequisites. The limitation across these works is that the generated plan is solved once and does not adapt to the pace the learner actually achieves, which is the feedback this project adds.

2.5 Self-Regulated-Learning Analytics

Self-regulated-learning (SRL) analytics grounds the choice of metrics for a planner that reports back to the learner. Alhazbi et al. [18] review 25 empirical studies on measuring SRL from trace data and find that most validated indicators relate to time-management skills, which grounds this project’s use of schedule adherence and regularity as its behavioural metrics. de Barba et al. [19] develop and validate an SRL learning-analytics rubric that maps validated SRL scales onto learning-management-system indicators in a scalable, explainable way.

Uzun et al. [20] show that a learner’s engagement with analytics feedback is associated with both SRL competence and course performance, which motivates the feedback-oriented design of this project’s dashboards. Butt et al. [21] apply a hybrid machine-learning and rough-set model to identify at-risk learners and trigger personalised interventions. These works supply validated, trace-based indicators of self-regulation, but they stop at the feedback step shown in Figure 2.5, and none feeds the SRL signal back into an adaptive plan.

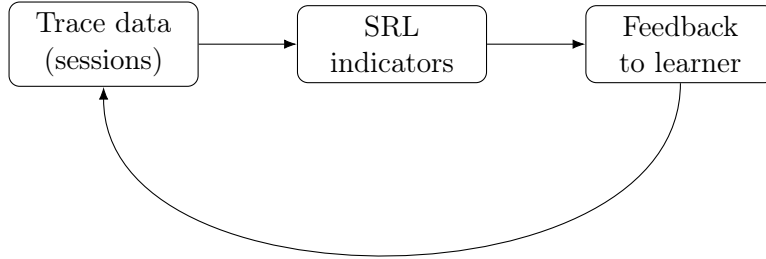


Figure 2.5: The self-regulated-learning analytics loop the reviewed work supports: trace data yields validated indicators that are fed back to the learner. The reviewed systems stop at feedback; this project extends the loop by feeding the signal into replanning.

2.6 Methodology for Sourcing the Surveyed Models

The literature above was assembled through a repeatable screening pipeline rather than by ad-hoc reading, so that the coverage of each theme is defensible. A raw candidate list of 107 Pillar-A digital object identifiers was collected from database and citation searches. Each candidate was enriched with metadata from CrossRef, OpenAlex, and arXiv, and scored against twelve weighted keyword clusters aligned with the project’s components, including Bayesian calibration, change-point detection, Gaussian-process regression, adaptive scheduling, and self-regulated learning. The score thresholded each paper into a strong, good, partial, or weak fit.

The score was treated as advisory rather than decisive. Final selection favoured topical coverage across sub-themes over raw score: one representative work was kept per sub-theme, and higher-scoring but redundant papers were dropped, while a lower-scoring paper was retained when it was the best available fit for a specific narrative role. The screening also rejected keyword false positives, where a paper matched a cluster keyword but was topically unrelated. This curation reduced the raw list to the set of works surveyed in this chapter.

2.7 Summary of Approaches

Table 2.1 summarizes the approach families across the components with their strengths and the limitations that motivate this work. The recurring pattern is that each technique is strong in isolation but disconnected from the others: none forms a closed loop from realised pace back to the plan.

Table 2.1: Summary of Pillar-A approach families, with strengths and limitations for this project.

Approach family	Strengths	Limitations for this project
Bayesian calibration [1, 2, 3, 4]	Works with sparse, per-learner data; interpretable; fair across groups	Targets correctness, not planned-versus-actual pace
Change-point detection [5, 6, 7, 8]	Detects behavioural shifts early; well-founded in statistical process control	Detects only; prescribes no corrective replanning
Gaussian-process projection [9, 10, 11, 12, 13]	Calibrated uncertainty; degrades gracefully with little data	Applied cross-sectionally, not to longitudinal burn-up
Adaptive scheduling [14, 15, 16, 17]	Fast, prerequisite-aware; adaptation shown to help	Generated plan is static; no feedback from realised pace
SRL analytics [18, 19, 20, 21]	Validated trace-based indicators of self-regulation	Measures behaviour; not tied to adaptive replanning

The limitations that motivate this project follow from the table. First, calibration, change detection, projection, and scheduling are each studied in isolation, and no reviewed system integrates them into a single connected loop. Second, behavioural monitoring detects shifts but prescribes no corrective replanning, and the scheduling work is static and does not adapt to realised pace. Third, the Gaussian-process work is cross-sectional, so calibrated learning-curve projection is not applied to ongoing self-directed study. Fourth, self-regulated-learning analytics supplies validated indicators but stops at measurement, without feeding the signal back into the plan.

This project addresses these gaps by extending the predict-then-optimize architecture [14] into an adaptive loop for a single self-directed learner. It combines hierarchical Bayesian pace calibration with a context-aware shrinkage model [1], change-point detection of behavioural shifts surfaced as a recalibration prompt [5, 6], Gaussian-process progress projection with a calibrated uncertainty band [9], and capacity-based schedule regeneration [15], evaluated with the adaptive-versus-static paradigm [16] and self-regulated-learning metrics [18]. Phase II will extend this open-loop system with assessment-based verification of genuine learning, so that the pace signal driving the plan is checked against evidence of learning rather than self-reported time alone.

Chapter 3

System Requirements Specification

This chapter specifies the requirements of the Phase-I system. Section 3.1 states the hardware environment. Section 3.2 states the software environment. Section 3.3 lists the functional requirements as numbered items, and Section 3.4 lists the non-functional requirements.

3.1 Hardware Requirements

The system is designed to run on commodity hardware. A single learner produces a small volume of data, and every model trains and runs on a CPU, so no dedicated graphics processor is required. Table 3.1 lists the hardware environment.

Table 3.1: Hardware environment.

Component	Requirement
Processor	Any modern x86-64 or ARM CPU; no GPU required
Memory	8 GB RAM or higher
Storage	A few gigabytes for the container image and local data
Container runtime	Docker (via Colima on macOS) to run the intelligence service
Client device	Any device with a modern web browser

3.2 Software Requirements

The system is organised into three tiers: a local-first browser client, a Python intelligence service that hosts the heavier statistical models, and an offline research pipeline used to compare and select those models. Supabase provides authentication, the event database, and object storage that link the client and the service. Table 3.2 lists the software environment by tier.

Table 3.2: Software environment by tier.

Tier	Technologies
Client (single-page app)	React 19, Vite, TypeScript, Dexie over IndexedDB for local-first storage, date-fns
Intelligence service	Python 3.12, FastAPI, Uvicorn, NumPy and SciPy, PyJWT; packaged with Docker
Backend services	Supabase: authentication, a PostgreSQL event database, and object storage
Offline research pipeline	Python (uv workspace), NumPy, with pytest and ruff for testing and linting
Tooling and deployment	pnpm workspace, Vitest and Playwright for testing, Vercel for hosting

3.3 Functional Requirements

The functional requirements state what the Phase-I system does.

- FR-1** The system shall let a learner define a study plan from a set of materials, a deadline, and a weekly study capacity.
- FR-2** The system shall log study sessions, both from an in-application timer and from manual entry of a past session.
- FR-3** The system shall estimate the learner’s pace from the logged sessions using hierarchical Bayesian calibration.
- FR-4** The system shall detect a shift in the learner’s pace and surface a recalibration prompt for the learner to act on.
- FR-5** The system shall project the completion date with a calibrated uncertainty band rather than a single date.
- FR-6** The system shall regenerate the schedule from the learner’s realised capacity when the learner chooses to replan.
- FR-7** The system shall present the learner’s progress, including the burn-up curve, streak, and schedule adherence.
- FR-8** The system shall persist all data on the device and synchronize it to the cloud when a connection is available.

3.4 Non-Functional Requirements

The non-functional requirements state the qualities the system must exhibit.

- NFR-1 Offline-first operation.** The core logging loop shall function without network connectivity, and analytical results shall be cached locally for reuse.
- NFR-2 Commodity scale.** The system shall serve the individual-learner workload on commodity hardware without specialised accelerators.
- NFR-3 Per-learner isolation.** Each learner’s data shall be isolated, using a per-user local database and row-level security in the cloud database.
- NFR-4 Authenticated services.** Requests to the intelligence service shall carry a signed token that is verified against the identity provider before any model runs.

- NFR-5 Responsiveness.** Calibration and projection results shall return within a few seconds, with a fallback to the last cached result when the service is unreachable.
- NFR-6 Behavioural alignment.** Where an algorithm exists in both the client and the service, the two implementations shall be kept behaviourally aligned so that results do not depend on which one runs.

Chapter 4

Proposed Methodology

This chapter describes the methodology of the Phase-I system. Section 4.1 presents the adaptive planning loop that ties the four components together. Section 4.2 states the candidate models considered for each component and the model selected for the delivered system. Section 4.3 describes the offline comparison methodology used to make that selection. Section 4.4 records how the system design evolved during development, from a predict-then-optimize scheduler to a capacity-based booking model. Section 4.5 describes the resulting system architecture and the split between client-side and server-side computation.

4.1 The Adaptive Planning Loop

The planner is organised as an open loop over the learner’s own logged sessions. A session is logged, the learner’s pace is recalibrated from the accumulated history, the pace series is checked for a behavioural shift, the completion date is re-projected, and the schedule is regenerated when the projection diverges from the deadline. The loop is deliberately open rather than automatic: a detected shift raises a recalibration prompt, and the learner decides whether to replan. This keeps the learner in control of a plan that is theirs, and it avoids acting on a shift that the learner knows to be a one-off. Figure 4.1 shows the four components and the signal that passes between them.

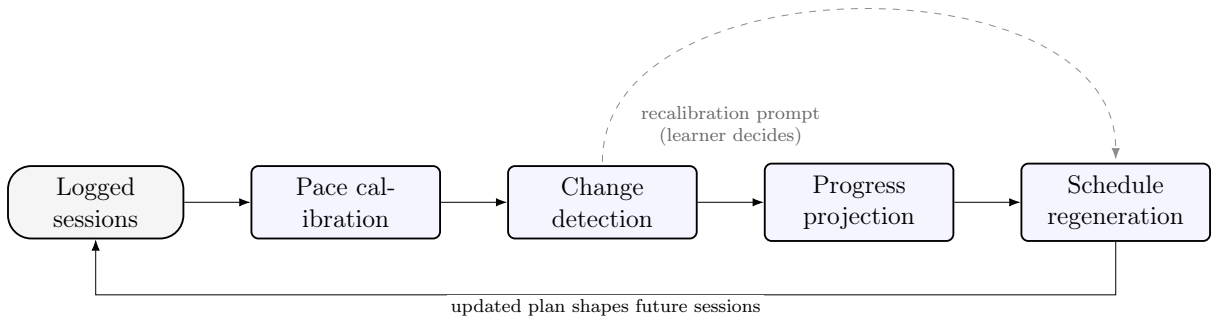


Figure 4.1: The open adaptive loop. Logged sessions drive pace calibration, change detection, and progress projection; a detected shift raises a recalibration prompt, and the learner decides whether to regenerate the schedule, which in turn shapes future sessions.

4.2 Candidate Models and Selected Approach

Each component was chosen by comparing candidate models offline rather than by assumption. This section states the candidates and the selected model; Section 4.3 describes how the comparison was run. Table 4.1 summarises the choices.

4.2.1 Pace Calibration

The calibration candidates were a simple moving average of the pace ratio, an exponentially weighted moving average, a pooled Bayesian estimate, and the hierarchical Bayesian model of Equation (2.1). Under rigorous held-out evaluation the structured candidates did not beat the hierarchical Bayesian baseline, which is reported honestly in Chapter 6. The model that did improve on the baseline is a context-aware shrinkage calibrator developed for this project. Let $\phi_t \in \mathbb{R}^d$ be a vector of observable context features for session t , covering the material role, the time of day, the day of week, proximity to the deadline, planned progress, and recency, and let $y_t = \log(a_t/p_t)$ be the log pace ratio. The calibrator fits weights w by ridge-regularised least squares shrunk toward a population prior w_0 rather than toward zero,

$$\hat{w} = \arg \min_w \sum_t (y_t - w^\top \phi_t)^2 + \lambda \|w\|^2 + \gamma \|w - w_0\|^2, \quad (4.1)$$

where λ is the ridge strength and γ the shrinkage strength, both fixed at values validated offline. The pace multiplier for a context ϕ is $\exp(\hat{w}^\top \phi)$. Shrinking toward a population prior makes the model behave sensibly when a learner has logged few sessions and converge to learner-specific behaviour as sessions accumulate. The delivered calibrator is an ensemble of two such models seeded with different population priors and combined by leave-one-out-weighted averaging.

4.2.2 Change Detection

The detection candidates were the one-sided CUSUM chart of Equation (2.2), an exponentially weighted moving-average control chart, a critical-slowness detector, the Page-Hinkley test, Bayesian online change-point detection, and ADWIN. The delivered system uses the single-arm CUSUM. As Chapter 6 reports, the comparison is a robust null: no candidate improves on the established CUSUM and critical-slowness frontier, and CUSUM is the fastest detector, which suits raising a human-confirmed recalibration prompt.

4.2.3 Progress Projection

The projection candidates were linear extrapolation of cumulative progress, Gaussian-process regression under the kernel of Equation (2.4), a composite that falls back to an analytic estimate when the learner has too few sessions for the Gaussian process to be trusted, and a split-conformal correction of the interval width. The delivered system uses the Gaussian process with the cold-start composite, which was validated as an improvement on the plain Gaussian process for low-data plans. The split-conformal interval correction is recorded in Chapter 6 as a validated refinement not yet shipped.

4.2.4 Schedule Generation

The scheduling candidates were a rule-based baseline, a dynamic-programming capacity allocator of the form in Equation (2.5), a greedy prerequisite-aware packer, and a local-search repair heuristic. On the synthetic data the dynamic-programming allocator minimised deadline drift. The delivered system does not use it. As Section 4.4 records, the day-by-day packing formulation these candidates share proved brittle in practice and worked against continuous replanning, so the system was redesigned around a capacity-based booking model. Scheduling is therefore reported in Chapter 6 as an evaluated component whose result informed a design decision, not as a shipped optimiser.

Table 4.1: Candidate models per component and the model selected for the delivered Phase-I system.

Component	Candidates compared	Selected (delivered)
Pace calibration	Moving average, EWMA, pooled Bayesian, hierarchical Bayesian, context-aware shrinkage	Hierarchical Bayesian with the context-aware shrinkage calibrator
Change detection	CUSUM, EWMA control chart, critical slowing down, Page-Hinkley, Bayesian online change-point, ADWIN	Single-arm CUSUM (robust null: nothing beats the CUSUM/CSD frontier)
Progress projection	Linear extrapolation, Gaussian process, GP with cold-start composite, split-conformal correction	Gaussian process with the cold-start composite
Schedule generation	Rule-based, dynamic-programming capacity, greedy prerequisite-aware, local-search repair	None shipped; replaced by capacity-based booking (Section 4.4)

4.3 Comparison Methodology

The comparison rests on synthetic data with known ground truth, so that each component can be scored against a truth that real learner logs do not provide. A synthetic generator produces study histories in which the learner’s true pace, the timing and size of any regime shift, and the completion behaviour are planted and therefore known. The generator’s parameters were calibrated against a large public learning-management-system dataset so that the synthetic regime resembles observed learner behaviour rather than an arbitrary one. Candidate models are evaluated on held-out learner archetypes that are absent from the set used to develop them, so that a win reflects generalisation rather than fitting to the evaluation cases. Figure 4.2 shows the pipeline.

Each component is scored on metrics appropriate to it, defined in Table 4.2. Every reported comparison carries a bootstrap confidence interval on the difference between a candidate and the incumbent, and a Holm or Benjamini–Hochberg correction is applied across the family of comparisons so that a win survives multiple-comparison scrutiny. An oracle baseline, which is given the planted ground truth, is included as an upper bound

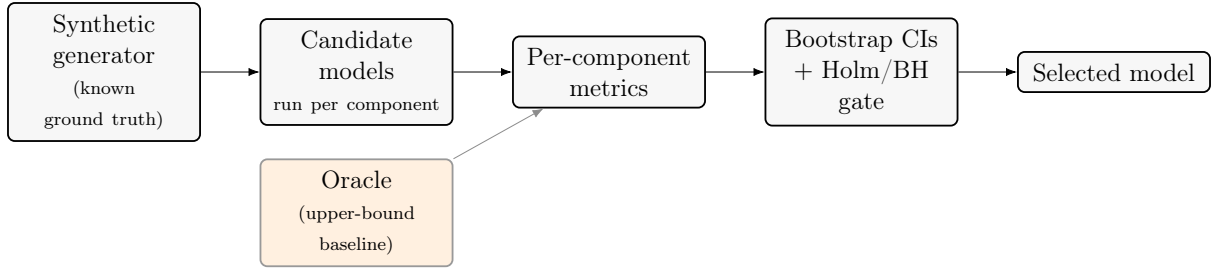


Figure 4.2: The offline comparison pipeline. The synthetic generator supplies ground truth; each candidate is scored on per-component metrics; a bootstrap confidence interval and Holm/Benjamini–Hochberg correction gate the result before a model is selected. The oracle is an upper-bound reference, not a deployable method.

on achievable performance; it is a reference ceiling and is never treated as a deployable method.

Table 4.2: Evaluation metrics by component.

Component	Metrics
Pace calibration	Mean absolute error of the recovered pace against the planted pace
Change detection	Detection latency after a planted shift, and the false-alarm rate
Progress projection	Coverage of the 95% interval against nominal, and point error in days
Schedule generation	Deadline drift and the number of capacity violations

4.4 System Design and Evolution

The system did not arrive at its current shape in one step, and the path is worth recording because it explains why the delivered scheduler differs from the one the comparison favoured.

The starting point was the predict-then-optimize architecture of the base paper [14], in which a predictive model estimates the learner’s pace and an optimizer turns the estimate into a schedule. The initial design realised this literally. The intelligence service ran calibration, detection, projection, and a constraint-based schedule generator, and the generator packed each material onto a specific calendar day, producing the dated slot grid shown in Figure 4.3. A replan re-ran the packer on the service and returned a new grid.

This design proved brittle in use. Pinning materials to fixed days created boundary and tie cases at the edges of the packing that were awkward to resolve, and the rigid day-by-day grid worked against the central goal of continuous replanning: every change to the learner’s pace forced a re-pack of the whole grid. The problem the packer solved, deciding exactly which material to study on which day, was also not the problem the learner actually had, which was knowing whether they were on track and what to do next.

The design was therefore revised. The scheduling question was re-examined on a revised synthetic regime that models the new interaction pattern, and the calibration,

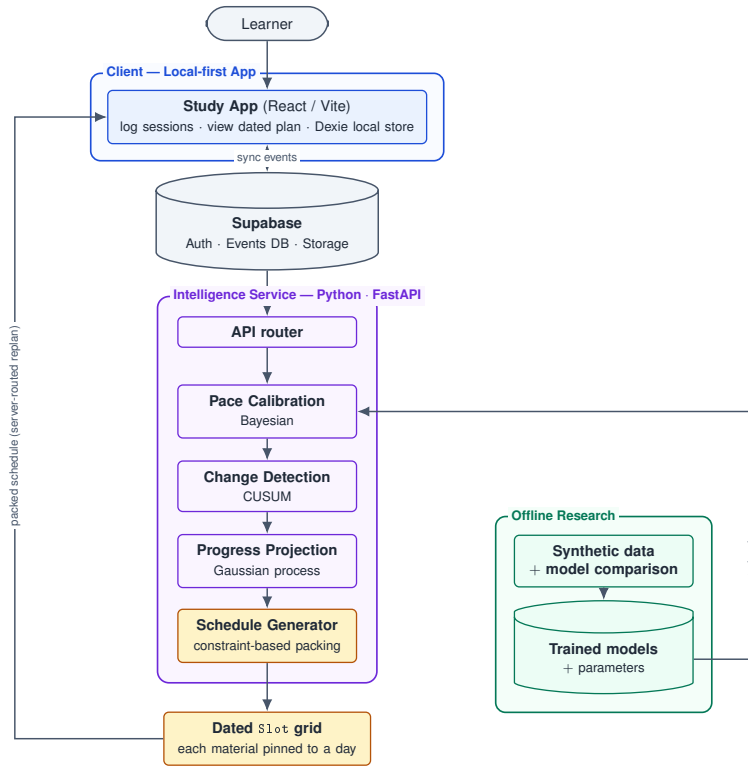


Figure 4.3: Initial design (predict-then-optimize). The intelligence service runs all four components, and the schedule generator packs each material onto a fixed day, returning a dated `Slot` grid; a replan re-runs the packer on the service.

detection, and projection models were confirmed to transfer to it. The prescriptive packer was retired in favour of a capacity-based booking model: the plan holds blank bookings, each carrying a date and a duration, and the learner picks a material at the start of a session. The engine’s job shrinks from packing a grid to estimating a capacity and projecting a finish date. Figure 4.4 shows the resulting architecture.

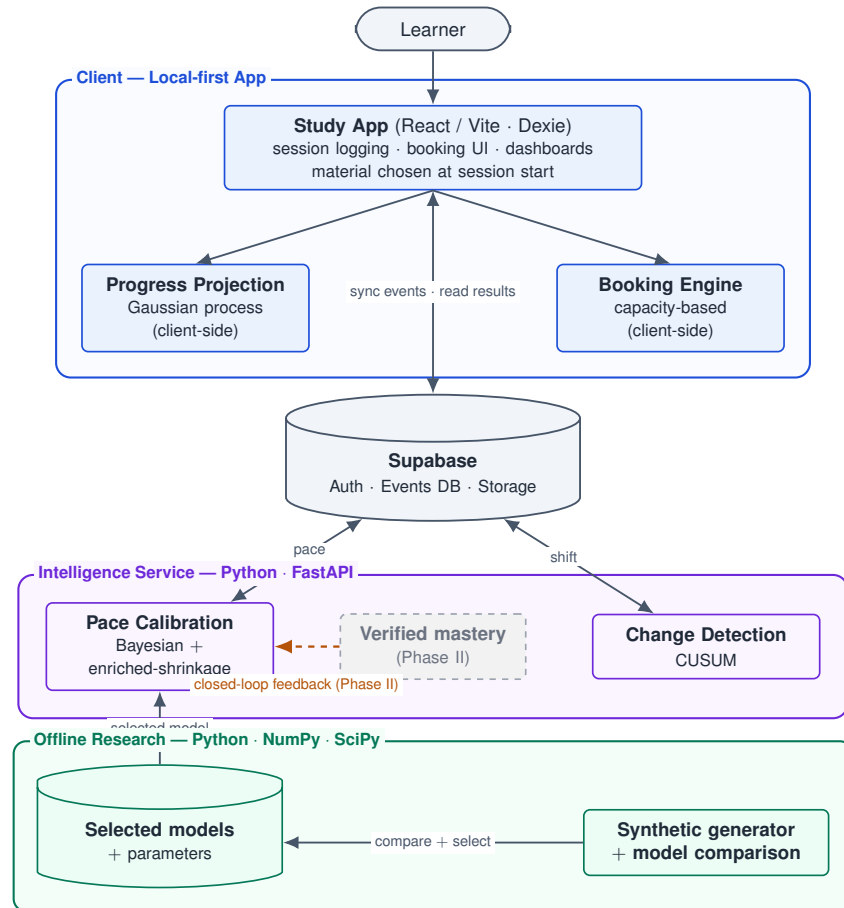


Figure 4.4: Revised design (capacity-based booking), the delivered system. Progress projection and the booking engine run client-side; Bayesian and enriched-shrinkage calibration and CUSUM detection run in the Python service; Supabase is the data hub; the offline research pipeline supplies the selected models. The closed-loop feedback from verified mastery is drawn dashed as Phase-II work.

4.5 System Architecture

The delivered system is organised into three tiers, shown in Figure 4.4, with computation deliberately split between the client and the service according to what each part needs.

The client is a local-first single-page application built with React and Vite, storing every event in a per-user IndexedDB database through Dexie. It owns the parts of the system that derive directly from the learner’s local event log: session logging, the booking user interface, the dashboards, the Gaussian-process progress projection, and the capacity-based booking engine. Running these client-side keeps the core loop responsive

and available offline. The intelligence service is a Python and FastAPI application that hosts the parts that fit a statistical model to accumulated data: Bayesian and enriched-shrinkage pace calibration and CUSUM change detection. The client calls the service over HTTP, and every request carries a signed token that the service verifies before any model runs. The division follows a simple rule: computation that fits a model to history runs on the service, and computation that derives a view from the local event log runs on the client. Supabase sits between the two as the data hub, providing authentication, the event database, and object storage, with each learner’s data isolated by row-level security. An offline research pipeline, described in Section 4.3, supplies the trained models and parameters that the calibrator uses. The closed-loop feedback from verified mastery back into calibration, shown dashed in Figure 4.4, is the principal Phase-II extension and is not part of the delivered Phase-I system.

Chapter 5

Implementation Details

This chapter describes how the system was built. Section 5.1 states the development environment. Section 5.2 describes the datasets and their sources. Section 5.3 describes the model implementation and how the client and service share it. Section 5.4 records the evaluation metrics and the steps taken for reproducibility. Section 5.5 presents the delivered application.

5.1 Development Environment

The project is a single repository with two overlapping workspaces. A pnpm workspace holds the TypeScript client and the shared client-side engines, and a uv workspace holds the Python packages, the intelligence service, and the offline research code. The two coexist so that the research code can import the same Python engines the service runs, which keeps the evaluated code and the shipped code identical. Table 5.1 lists the environment.

Table 5.1: Development environment: tools and libraries.

Concern	Technology
Client language and build	TypeScript, Vite, pnpm workspace
Client UI	React 19
Local-first storage	Dexie over IndexedDB
Intelligence service	Python 3.12, FastAPI, Uvicorn
Scientific stack	NumPy, SciPy
Service authentication	PyJWT, verifying Supabase-issued tokens
Containerisation	Docker via Colima
Python workspace	uv
JavaScript testing	Vitest, fast-check (property tests), Playwright (end-to-end)
Python testing and linting	pytest, ruff
Backend and hosting	Supabase (auth, database, storage), Vercel

5.2 Datasets and Their Sources

Because a single learner’s real log cannot supply a known ground truth, the comparison in Chapter 6 is run on synthetic data whose properties are planted and therefore known. Table 5.2 lists the datasets and the role each plays.

The synthetic generator produces study histories in which the learner’s true pace, the timing and size of any regime shift, and the completion behaviour are set explicitly. So that the synthetic regime resembles real behaviour rather than an arbitrary one, its parameters were calibrated against the Open University Learning Analytics Dataset (OULAD), a large public dataset of learner activity. The OULAD is used only to calibrate the generator’s realism; it is not used as a direct benchmark. A revised generator regime models the booking interaction pattern introduced by the redesign of Section 4.4, and the selected models were re-checked against it. Real single-learner logs are reserved for the longitudinal validation planned in Phase II and are not used in Phase I.

Table 5.2: Datasets used in Phase I and their sources.

Dataset	Source	Role
Synthetic study histories	Generated in-repository	Primary ground truth for the component comparison; plants pace, shifts, and learner archetypes
OULAD	Public (Open University)	Calibrates the synthetic generator’s realism regime; not a direct benchmark
Revised (decoupled) regime	Generated in-repository	Re-validates the selected models under the booking interaction model
Real single-learner log	The candidate’s own sessions	Phase-II longitudinal validation; not used in Phase I

5.3 Model Implementation

The algorithms are implemented once in behaviour and twice in code. The client-side engines are written in TypeScript, and a Python mirror of each engine is exposed through the intelligence service. The two implementations are kept aligned by a golden-fixtured parity harness: the TypeScript engines emit reference inputs and outputs, and a Python test replays each input and asserts numeric equality to a tight tolerance. The shared numeric constants, such as the Bayesian prior, the CUSUM slack and threshold factors, and the Gaussian-process length scale, are held identical on both sides.

The split between client and service follows the architecture of Figure 4.4. Computation that fits a statistical model to accumulated history runs on the service: Bayesian and enriched-shrinkage pace calibration and CUSUM change detection, reached over HTTP at a calibration endpoint that verifies a signed token before running. Computation that derives a view from the local event log runs in the browser: the Gaussian-process projection, the burn-up and streak summaries, and the capacity-based booking engine. The offline research harness imports the production Python engines as candidates rather than re-implementing them, so a result measured offline is a result about the shipped code. Listing 5.1 shows the shipped change detector, which realises Equation (2.2), and Table 5.3 lists the main packages.

Listing 5.1: The shipped one-sided CUSUM detector, realising Equation (2.2).

```
def run_cusum(series, mu0, k, h):
    """Return the index of the first alarm, or None if no shift is detected."""
    s = 0.0
    for t, x in enumerate(series):
        s = max(0.0, s + (x - mu0) - k)    # accumulate deviation, floored at zero
        if s > h:                          # threshold crossed -> shift detected
            return t
    return None
```

Table 5.3: Main packages and their roles.

Package / module	Role
packages/progress (TS)	Client engine: Gaussian-process projection, burn-up, streak, calibration mirror
packages/roadmap-engine (TS)	Client booking and roadmap generation
packages/py-progress (Py)	Python mirror: Bayesian and enriched-shrinkage calibration, CUSUM, Gaussian process
packages/py-roadmap-engine (Py)	Python mirror of the roadmap engine
services/intelligence (Py)	FastAPI service exposing calibration over HTTP, token-verified
research/comparison (Py)	Offline harness importing the production engines as candidates

5.4 Evaluation Metrics and Reproducibility

The metrics used to score each component are defined in Table 4.2 of Chapter 4: recovery error for calibration, detection latency and false-alarm rate for change detection, interval coverage and point error for projection, and deadline drift and capacity violations for scheduling. This section records how they are computed and how the results are made reproducible.

Every reported comparison is accompanied by a bootstrap confidence interval on the difference between a candidate and the incumbent, estimated by resampling learners with replacement. Across the family of comparisons a Holm or Benjamini–Hochberg correction is applied, so that a claimed win survives multiple-comparison scrutiny rather than arising from testing many candidates. A result is reported only when it passes a set of credibility checks that guard against degenerate or unstable cells. The pipeline is driven by fixed random seeds and a small set of make targets, so that the dataset, the comparison, and the figures regenerate deterministically, and each generated figure and table carries a provenance stamp recording the run that produced it.

5.5 The Phase-1 Application

The selected models are not left as an offline result. They are realised in a working, local-first web application for a single self-directed learner, which this section presents in full. The application is organised around four capabilities, and the section follows them. Section 5.5.1 covers plan creation, where onboarding turns a deadline, a weekly capacity,

and a set of materials into a booked schedule. Section 5.5.2 covers study execution, the session runtime under which the work is done and recorded. Section 5.5.3 covers progress and adaptation, the dashboards that report calibrated pace and projected completion and the screen that adjusts the plan. Section 5.5.4 covers plan lifecycle, the calendar and dashboard that track a plan from draft to completion. Section 5.5.5 describes the local-first data architecture underneath all four. Every screenshot in this section is captured from the running application.

5.5.1 Plan Creation: Onboarding and Roadmap Generation

A new learner is taken through a four-step onboarding wizard that collects the planner’s inputs and produces an initial schedule. The first step records the target deadline and an optional statement of purpose. The second records a weekly study capacity, divided by default in a 60/40 ratio between weekdays and weekends over a chosen set of study days. The third collects the study materials: a material is added by pasting a link, which a lightweight classifier resolves into a YouTube video, a playlist, an article, or a manual entry, and each material is assigned a role of anchor, foundation, or practice that later governs its ordering. The fourth step presents a read-only summary before the plan is committed. Figure 5.1 shows the generated plan at the close of onboarding.

From these inputs the client-side roadmap engine generates the schedule. Rather than packing each day with a prescribed material, as the retired constraint-based design did (Section 4.4), the engine lays capacity-sized bookings across the study days between the start date and the deadline, and leaves the specific material to be chosen at the start of each session. The wizard writes its state to the local event log on every step, so a learner who closes the tab midway resumes where they stopped instead of starting over. Committing the plan emits the events the rest of the system reads: a `MaterialAdded` event for each material, a `RoadmapCreated` event for the plan, a `SessionBooked` event for each booking, and an `OnboardingCompleted` event that admits the learner to the main application.

5.5.2 Study Execution: the Session Runtime

When it is time to study, the learner starts a session from the plan and chooses the material to work on at that moment. The session runs under a timer with optional Pomodoro intervals of fifty working minutes separated by ten-minute breaks. The interval clock advances on wall-clock time, so a break that is due still arrives even after the timer has been paused. A YouTube material plays in an embedded player whose position is tracked, so that a multi-video playlist resumes at the point it reached, while an article opens in a separate tab and is timed alongside. Figure 5.2 shows a session in progress.

The runtime is governed by a small state machine over idle, active, and paused states that also recognises a stale session, such as a timer left running overnight or a tab hidden for several minutes, and offers a recovery path rather than recording a spurious multi-hour session. When a session ends it records the minutes actually worked, the number of pauses, the Pomodoro blocks completed, and the position reached in each material, and this record is what later feeds the pace calibration and the progress views. A session can also be flagged as exceptional, marking an atypical day so that it does not distort the pace estimate.



Figure 5.1: Onboarding: the generated booking plan produced from the learner's deadline, weekly capacity, and materials.

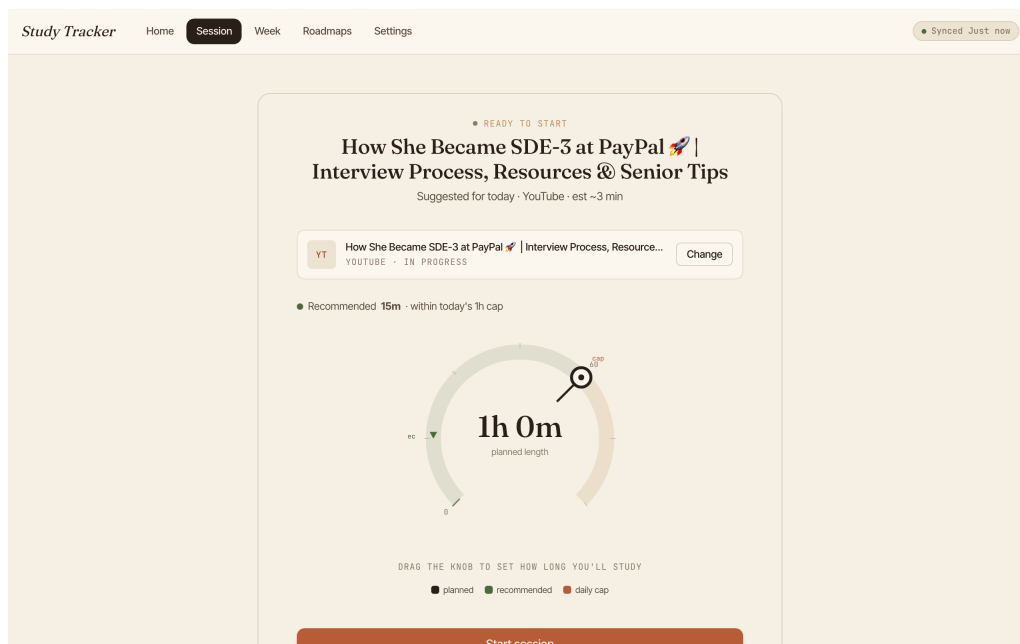


Figure 5.2: A study session in progress, with the timer and material chosen at the start of the session.

5.5.3 Progress and Adaptation: Home, Week, and Replan

Between sessions the learner sees how the plan is tracking. The Home dashboard (Figure 5.3) summarises the current state in a single view: the next booking to act on, the current and best study streak, the calibrated pace, and the projected finish date. The calibrated pace is produced by the intelligence service. The browser sends the session history to a token-verified calibration endpoint, and the Python engine returns the hierarchical-Bayesian estimate refined by the context-aware shrinkage calibrator of Equation (4.1), together with per-role multipliers. The projection and the streak, by contrast, are computed in the browser from the local event log.

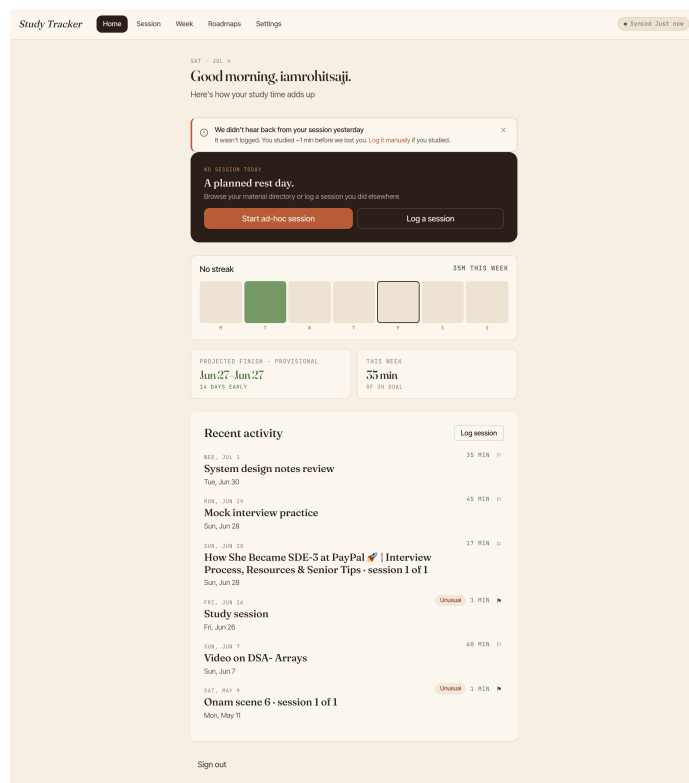


Figure 5.3: The Home dashboard: up-next booking, streak, calibrated pace, and projected finish date.

The Week view (Figure 5.4) reports a single verdict for the week, ahead, on track, or behind, above a daily-minutes chart and a burn-up projection. The burn-up plots the cumulative minutes actually logged against the planned ramp and overlays the Gaussian-process mean with its 95% band, falling back to an analytic estimate while fewer than five sessions have accumulated so that the early projection stays stable (Section 4.2). The point at which the band crosses the target line is the projected completion, read against the deadline.

When realised pace diverges from the plan, the system does not silently rewrite the schedule. Change detection runs server-side on the same calibration call, and a divergence surfaces as a recalibration prompt that the learner can accept, acknowledge, or defer; the plan is changed only on the learner's instruction, which keeps the loop open (Section 4.1). The Replan screen (Figure 5.5) offers three independent levers for that change: extend or shorten the deadline, raise or lower the weekly hours, or drop and trim materials. Each lever shows its effect on the projected finish date as it is moved, so the trade is

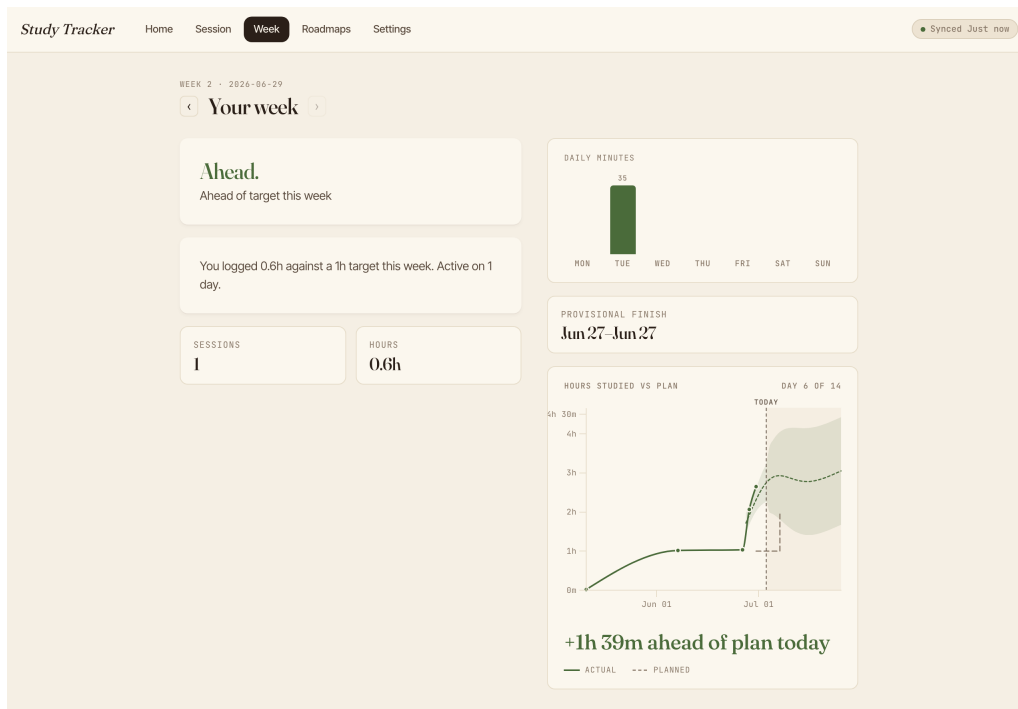


Figure 5.4: The Week view: the week’s verdict, the daily-minutes chart, and the burn-up projection.

visible before it is committed. Confirming a replan re-lays the bookings and records a `RoadmapReplanned` event that preserves the plan’s identity, so its history remains intact.

5.5.4 Plan Lifecycle: the Roadmap Calendar and Roadmaps Dashboard

A plan is more than its current week. The Roadmap calendar (Figure 5.6) shows the whole plan as a month grid, with study days tinted and each booking placed on its day and coloured by material role. Selecting a day opens a sheet in which the learner can add, edit, or clear a booking and mark how far through a material they have reached. A booking’s status, whether unstated, in progress, completed, trimmed, or interrupted, is derived from the logged sessions rather than stored, so it stays consistent with what actually happened.

The Roadmaps dashboard (Figure 5.7) tracks plans across their lifecycle. The active plan is shown with its date range and progress, an unfinished onboarding draft is offered for resumption, and completed or abandoned plans are kept in a history that can be re-opened read-only. Because a replan preserves the plan’s identity, a plan and its successive replans collapse into a single entry rather than appearing as several unrelated plans.

5.5.5 The Local-First Data Architecture

The four capabilities above rest on a local-first, event-sourced data layer. Every action the learner takes is recorded as an immutable event in a log held in the browser’s IndexedDB, and every view, whether a dashboard, the calendar, or a projection, is derived by replaying that log rather than by reading mutable records. Because the log is local, the core loop stays usable without a network connection: reads and writes complete against the local

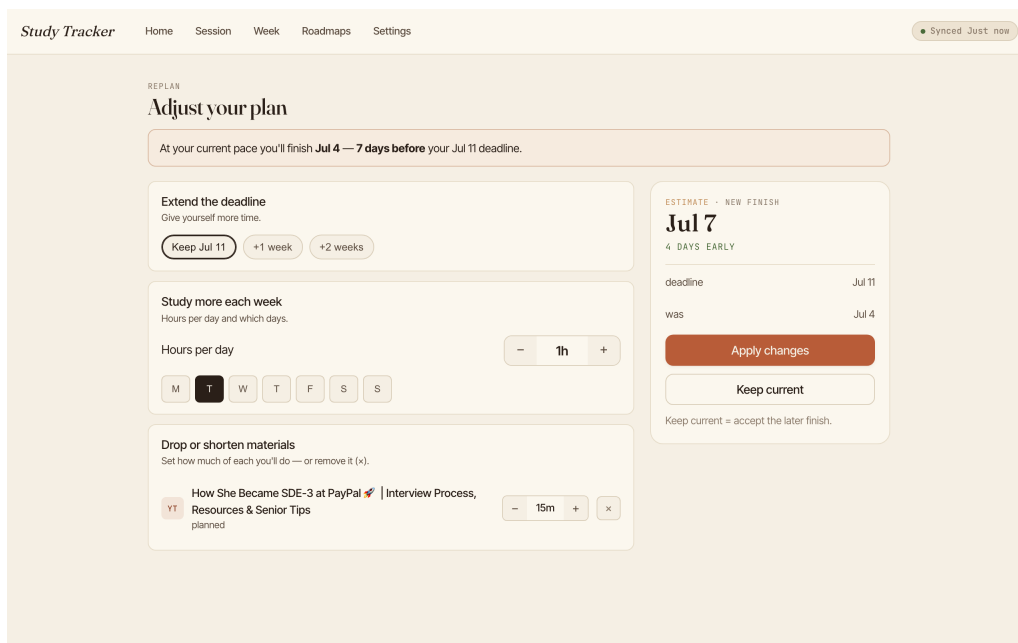


Figure 5.5: The Replan screen: three levers to adjust the plan when the projection diverges from the deadline.

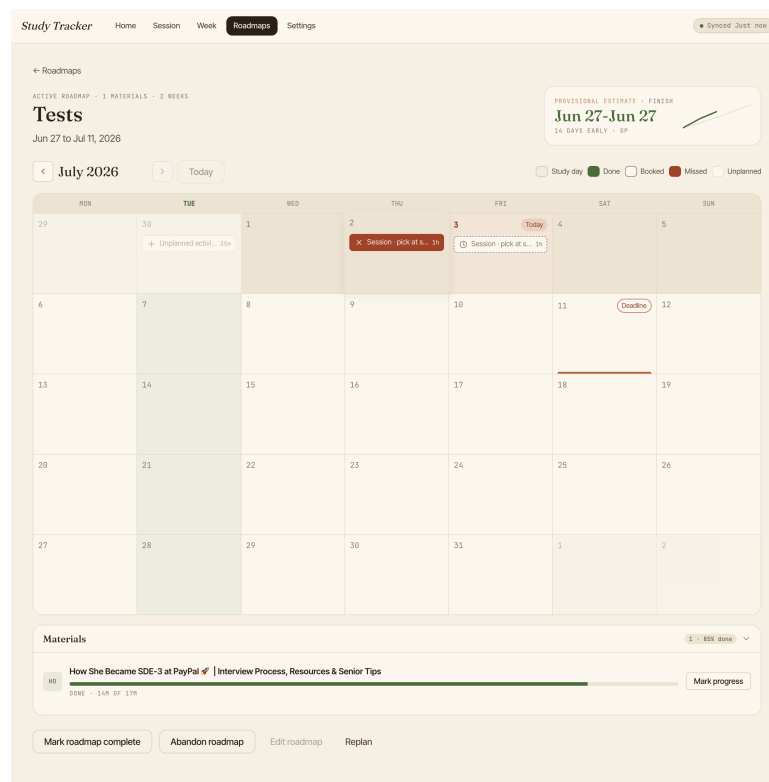


Figure 5.6: The Roadmap calendar: the plan shown as a month grid of bookings.

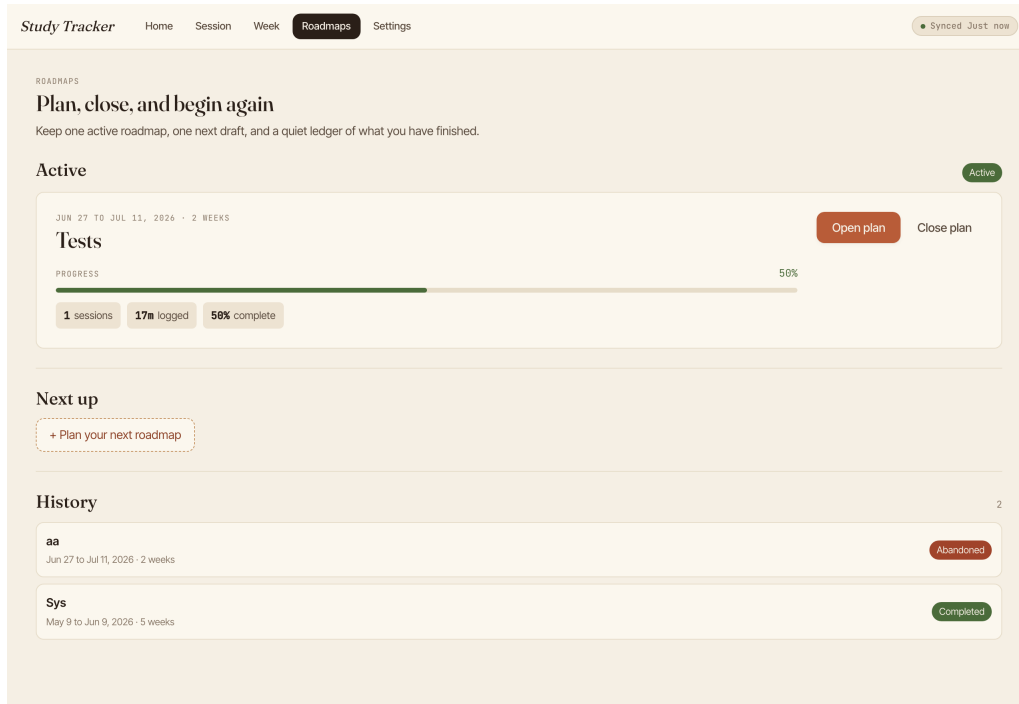


Figure 5.7: The Roadmaps dashboard: the active plan, drafts, and history over their lifecycle.

store and reconcile with the cloud when a connection returns. Each signed-in learner has a separate local database keyed by their identity, so two accounts sharing a browser never see each other's data and neither loses its history on sign-out.

Durability to the cloud is provided by a write-ahead queue. Events are enqueued locally and flushed to a Supabase table, and a periodic snapshot of the log is uploaded to object storage, so that a new device restores the full history from the snapshot and then replays only the events recorded since it. In Phase 1 the plan's own events, its creation, its bookings, and any replan, synchronise to the cloud through this queue, while extending the same durability to the stream of events emitted during a live study session is a hardening task carried into Phase II. The division of computation between this client and the intelligence service follows the architecture of Section 4.5 and Figure 4.4: model fitting over accumulated history runs in the service, and every view derived from the local log runs in the browser.

Chapter 6

Results and Discussion

This chapter reports the offline comparison for each Pillar-A component and discusses what the numbers mean for the delivered system. All results are computed on the synthetic ground-truth data described in Section 5.2, over 200 seeds and held-out learner archetypes, with bootstrap confidence intervals and Holm/Benjamini–Hochberg correction as described in Section 4.3. Each component is reported for both the initial (frozen) synthetic regime and the revised (decoupled) regime that matches the booking model, so that the transfer of a result across the redesign is visible. Throughout, the oracle is the upper-bound reference that is given the planted ground truth; it is never a deployable method. Section 6.1 covers pace calibration, Section 6.2 change detection, Section 6.3 progress projection, Section 6.4 schedule generation, and Section 6.5 discusses the results together.

6.1 Pace Calibration

Table 6.1 reports the mean recovery error of the pace estimate against the planted pace, by history-length band. The reading is deliberately honest: on this retrospective metric the structured Bayesian variants do not separate from the hierarchical-Bayesian incumbent, and the incumbent is in fact the strongest deployable method at the longest histories. The context-aware enriched-shrinkage calibrator improves on the incumbent on short histories, where its shrinkage toward a population prior helps most, but it is worse at the maximum band. This is an honest null for the structured calibrators on recovery error.

Table 6.1: Pace calibration: mean recovery error (lower is better) by history-length band, frozen regime. The oracle is the upper-bound reference.

Method	Short	Medium	Long
Oracle (reference ceiling)	0.000	0.000	0.000
Enriched-shrinkage (selected)	0.049	0.049	0.056
Hierarchical Bayesian (incumbent)	0.079	0.046	0.026
Moving average	0.094	0.100	0.090
EWMA	0.115	0.124	0.107

The reason the enriched-shrinkage calibrator was nonetheless selected is that recovery error is not the quantity the planner acts on. The planner needs a *prospective* pace estimate for the next session, especially when the learner has logged little history. On

that prospective metric the enriched-shrinkage calibrator is a robust improvement over the incumbent across every band and both regimes, and the improvement survives Holm correction (Table 6.2). It is to be noted that this is exactly the low-data regime in which a new learner’s plan first needs to adapt, which is why the prospective advantage was judged to matter more than the retrospective null.

Table 6.2: Enriched-shrinkage minus hierarchical-Bayesian on the prospective context-prediction error (negative favours enriched-shrinkage), with 95% bootstrap intervals.

Band	Frozen Δ [95% CI]	Decoupled Δ [95% CI]
Short	−0.0148 [−0.0168, −0.0127]	−0.0145 [−0.0155, −0.0136]
Medium	−0.0262 [−0.0282, −0.0243]	−0.0268 [−0.0277, −0.0258]
Long	−0.0300 [−0.0316, −0.0285]	−0.0307 [−0.0316, −0.0299]

6.2 Change Detection

Table 6.3 reports detection latency and false-alarm rate by shift type on the frozen regime. The comparison is a robust null in the sense that no exotic detector improves on the established CUSUM and critical-slowness frontier. CUSUM detects fastest, at roughly one and a half sessions after a shift, at the cost of the highest false-alarm rate; the critical-slowness and Page-Hinkley detectors trade latency for far fewer false alarms; the exponentially weighted control chart and the Bayesian online change-point detector are markedly slower; and the ADWIN detector fails to fire at all on this data. Under the combined score that penalises both late detection and false alarms, CUSUM wins on drift shifts in both regimes and is competitive on step shifts, where Page-Hinkley edges it in the frozen regime and CUSUM regains the lead in the decoupled regime.

Table 6.3: Change detection: mean detection latency (sessions, lower is better) and false-alarm rate by shift type, frozen regime. ADWIN did not fire on this data.

Detector	Step shift		Drift shift	
	Latency	False-alarm	Latency	False-alarm
CUSUM (selected)	1.43	0.312	1.36	0.296
Critical slowing down	5.16	0.083	4.10	0.049
Page-Hinkley	3.03	0.149	2.39	0.139
EWMA control chart	11.95	0.039	10.68	0.024
Bayesian online change point	30.59	0.009	19.43	0.011
ADWIN	—	0.000	—	0.000

The delivered system ships the single-arm CUSUM. Its higher false-alarm rate is acceptable here because a detected shift raises a recalibration prompt rather than triggering an automatic replan (Section 4.1): a false alarm costs the learner a dismissal, not a wrongly regenerated plan, whereas a late detection costs adaptation time. Fast detection with human confirmation is the better trade for this design.

6.3 Progress Projection

Table 6.4 reports the interval coverage and point error of the projection by history-length band, for both regimes. The point error is the same for the plain Gaussian process and the conformal method because the conformal step only rescales the interval width, not the point estimate. The headline result is a coverage failure of the raw Gaussian-process interval: against a nominal 95%, the plain interval covers only 19% to 49% of the time, and the shortfall worsens as the horizon lengthens. The split-conformal correction restores coverage to between 94% and 99%, at the same point error. Figure 6.1 shows the gap.

Table 6.4: Progress projection: interval coverage (nominal 0.95) and point error in days, by band. Point error is shared between the plain and conformal intervals.

Band	Coverage (frozen)		Coverage (decoupled)		Point error (days, frozen)
	Plain GP	Conformal	Plain GP	Conformal	
Short	0.438	0.969	0.494	0.989	2.15
Medium	0.291	0.990	0.277	0.988	9.45
Long	0.187	0.937	0.183	0.988	20.17

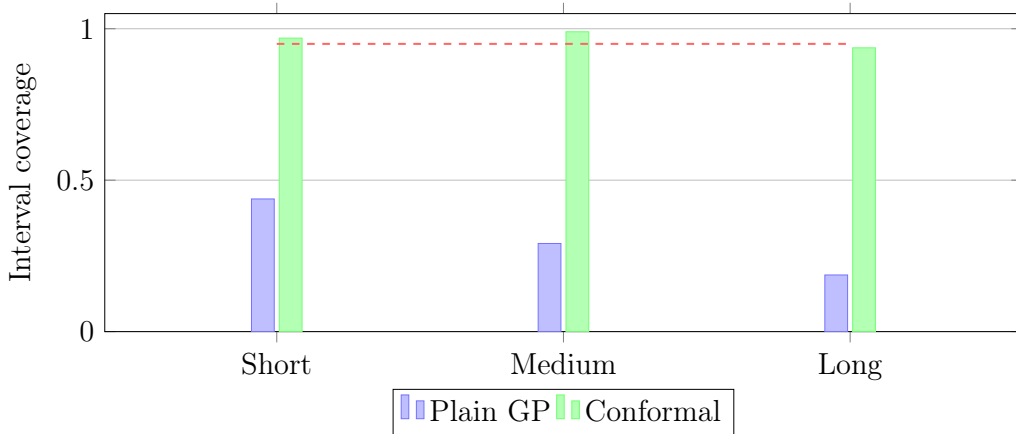


Figure 6.1: Interval coverage against the nominal 0.95 (dashed) by history-length band, frozen regime. The plain Gaussian-process interval under-covers badly; the split-conformal correction restores near-nominal coverage.

The delivered system ships the Gaussian process with a cold-start composite that falls back to an analytic estimate when the learner has too few sessions for the process to be trusted; on the decoupled regime this composite improves cold-start coverage over the raw Gaussian process, for example lifting short-history coverage from 0.19 to 0.44. The split-conformal correction, which is the fix for the data-rich under-coverage in Table 6.4, is validated here but is recorded as a follow-up not yet shipped; the delivered projection therefore reports an honest but under-covering interval past the cold-start threshold, a limitation returned to in Section 6.5 and Chapter 7.

6.4 Schedule Generation

Table 6.5 reports deadline drift and capacity violations by material mix on the reality-matched regime. The dynamic-programming capacity allocator minimises deadline drift

in every mix, and the margin over the greedy incumbent grows sharply with the number of interleaved material roles: on the full anchor-plus-foundation-plus-practice mix the greedy incumbent drifts by roughly 39 days while the dynamic-programming allocator drifts by about 8, a difference of -30.93 days $[-31.70, -30.19]$. All candidates respect the prerequisite ordering and, apart from a negligible rate for the rule-based baseline, incur no capacity violations.

Table 6.5: Schedule generation: mean absolute deadline drift (days) by material mix, reality-matched regime. Capacity violations were zero for the selected allocator.

Material mix	DP capacity	Greedy incumbent	Δ drift [95% CI]
Anchor	1.73	3.11	$-1.21 [-1.71, -0.72]$
Anchor + foundation	2.35	3.65	$-1.07 [-1.58, -0.55]$
Anchor + practice	3.91	5.94	$-2.03 [-2.71, -1.39]$
Anchor + foundation + practice	8.30	38.98	$-30.93 [-31.70, -30.19]$

This is a genuine win for the dynamic-programming allocator, and it is the one component result the delivered system does not act on. As recorded in Section 4.4, the day-by-day packing formulation these schedulers share was retired during development in favour of the capacity-based booking model, because rigid packing was brittle at day boundaries and worked against continuous replanning. The scheduling comparison is therefore reported as an evaluated result that informed a design decision, not as the behaviour of the shipped scheduler.

6.5 Discussion

Read together, the results separate into honest nulls, real wins, and validated refinements not yet shipped. Calibration is a null on retrospective recovery error and a real, Holm-surviving win for the enriched-shrinkage calibrator on the prospective, short-history regime the planner actually depends on. Detection is a robust null: nothing beats the CUSUM frontier, and the shipped CUSUM is the fastest detector, which suits a human-confirmed prompt. Projection produces the clearest finding of the chapter, that the raw Gaussian-process interval under-covers severely and that a split-conformal correction fixes it; the correction is validated but deferred, so interval calibration past the cold-start threshold is a known limitation. Scheduling is a real win for a method the system deliberately does not ship, because the problem it solved was removed by the redesign.

Two cross-cutting points support the credibility of these results. First, the selected models transfer across the two synthetic regimes: the enriched-shrinkage advantage on prospective error and the projection coverage failure both hold from the frozen regime to the decoupled regime that matches the booking model, so the findings are not artefacts of one data-generating process. Second, all of this rests on synthetic data with known ground truth; the synthetic setting is what makes latency, coverage, and recovery measurable at all, but it is not a substitute for real use. Validation on a single learner’s own logged sessions is deferred to Phase II, as set out in Chapter 7.

Chapter 7

Conclusion and Future Work

This chapter concludes Phase I and sets out the work planned for Phase II. Section 7.1 summarises what was built and found. Section 7.2 describes the Phase-II extensions.

7.1 Conclusion

This report has presented Phase I of an adaptive study-planning system for self-directed learners. The system estimates a learner’s pace with hierarchical Bayesian calibration and a context-aware enriched-shrinkage model, detects behavioural shifts with CUSUM, projects the completion date with a Gaussian process and a cold-start composite, and regenerates the plan with a capacity-based booking model. These components are realised in a working, local-first web application, with the pace calibration and change detection served by a Python intelligence service and the projection and booking engine running in the browser.

Each component was compared offline against established baselines on synthetic data with known ground truth, using held-out learner archetypes, bootstrap confidence intervals, and multiple-comparison correction. The evaluation was reported honestly rather than selectively. Pace calibration is an honest null on retrospective recovery error, where the structured Bayesian variants do not separate from the incumbent, but the enriched-shrinkage calibrator is a Holm-surviving improvement on the prospective, short-history prediction the planner depends on. Change detection is a robust null: no candidate beats the CUSUM frontier, and the fastest detector, CUSUM, is the one shipped because detection feeds a human-confirmed prompt. Progress projection produced the clearest finding, that the raw Gaussian-process interval under-covers severely and that a split-conformal correction restores near-nominal coverage. Schedule generation is a genuine win for a dynamic-programming allocator that the system deliberately does not ship, because the day-by-day packing problem it solved was removed by the move to a booking model.

The contribution of Phase I is an integrated, open-loop adaptive planner that extends the predict-then-optimize architecture [14] into a loop over a single learner’s own logged behaviour, together with a rigorous and honest account of which components improve on their baselines, which do not, and which improvements are validated but not yet shipped.

7.2 Future Work

Phase II will close the loop and add assessment-based verification of genuine learning, which is the principal novelty of the project. Three lines of work follow.

The first is verified learning. The pace signal that drives the plan is currently taken from self-reported study time, which is known to be gameable [28]. Phase II will generate concept-tagged assessments from the learner’s own study material using large language models [24, 25], tag each item to a knowledge component [26], grade responses automatically [27], and estimate genuine concept mastery with cold-start knowledge tracing suited to a single learner with little history [22, 23]. The verified-mastery estimate will be fed back into pace calibration, closing the loop shown dashed in Figure 4.4, so that the plan responds to evidence of learning rather than to logged time alone. Review timing will draw on spaced-repetition and personalised forgetting models [30, 29].

The second is to ship the refinements that Chapter 6 validated but that Phase I did not deliver, in particular the split-conformal correction of the projection interval, which fixes the under-coverage of the raw Gaussian-process interval past the cold-start threshold.

The third is longitudinal validation on real data. Phase I rests on synthetic ground truth; Phase II will validate the system on a single learner’s own logged sessions over time, comparing closed-loop adaptive planning against the static open-loop baseline on schedule adherence and measured learning gain.

Bibliography

- [1] Chen, et al., “A Hierarchical Bayesian Model of Adaptive Teaching,” *Cognitive Science*, 2024.
- [2] Zanellati, A., et al., “Hybrid Models for Knowledge Tracing: A Systematic Review,” *IEEE Transactions on Learning Technologies*, 2024.
- [3] Zambrano, A. F., et al., “Investigating Algorithmic Bias in Bayesian Knowledge Tracing,” *Proc. Learning Analytics and Knowledge (LAK)*, ACM, 2024.
- [4] Mai, N. T., Cao, W., Liu, W., “Interpretable Knowledge Tracing via Transformer–Bayesian Hybrid Networks,” *Applied Sciences*, doi:10.3390/app15179605, 2025.
- [5] Saqr, M., et al., “Early Warning Signals Before Dropping Out: Critical Slowing Down Indicators,” *Proc. Learning Analytics and Knowledge (LAK)*, ACM, 2026.
- [6] Qiao, et al., “Machine-Learning Approaches for Statistical Process Control: A Systematic Review,” *Entropy*, 2026.
- [7] Boca de Giuli, et al., “Monitoring and Continual Learning of Dynamical Models,” *Proc. European Control Conference (ECC)*, IEEE, 2024.
- [8] Cheng, J., Yang, Z.-Q., Cao, J., “Modeling Behavior Change for Multi-model At-Risk Students Early Prediction,” *Proc. International Symposium on Educational Technology (ISET)*, IEEE, doi:10.1109/iset61814.2024.00020, 2024.
- [9] Pérez-Suay, A., et al., “Predicting Student Performance from Moodle Logs with Gaussian Processes,” *IEEE-RITA*, 2024.
- [10] Chen, and Yao, “Predicting Academic Achievement with Black-Hole-Optimized Gaussian Process Regression,” *Scientific Reports*, 2025.
- [11] Lin, et al., “Scaling Gaussian Processes for Learning-Curve Prediction with Latent Kronecker Structure,” arXiv preprint, 2024.
- [12] Huang, K., Wang, C., “A Gaussian Process Classifier Integrating Meta-Heuristic Optimisers to Predict Student Performance,” *International Journal of Information and Management Sciences*, doi:10.1504/ijims.2025.149393, 2025.
- [13] Kayande, D., Kukreja, S., “An Integrated Multi-Modal Machine-Learning Framework for Real-Time Student Engagement Evaluation,” *MethodsX*, doi:10.1016/j.mex.2025.103588, 2025.

- [14] Islam, M., et al., “Predicting Student Performance and Optimizing Study Schedule using Deep Learning and Dynamic Programming,” *Proc. International Conference on Computer and Information Technology (ICCIT)*, IEEE, 2024.
- [15] Fabregar, and Amorado, “A Study Plan Recommender Based on a Greedy Algorithm,” *Proc. International Conference on Electrical Engineering and Informatics (EECSI)*, IEEE, 2025.
- [16] Pagano, et al., “Efficiency of Adaptive Learning Paths,” *Interactive Technology and Smart Education*, 2026.
- [17] Liu, X., Li, X., Chen, J., “Adaptive Learning Path Planning System Based on Knowledge Graph and AI Integration,” *Proc. International Conference on Educational Technology (ICET)*, IEEE, doi:10.1109/icet67421.2025.11380560, 2025.
- [18] Alhazbi, S., et al., “Using Learning Analytics to Measure Self-Regulated Learning: A Systematic Review,” *Journal of Computer Assisted Learning*, 2024.
- [19] de Barba, P. G., Oliveira, E. A., English, N., “Development and Validation of a Learning Analytics Rubric for Self-Regulated Learning,” *Educational Technology Research and Development*, doi:10.1007/s11423-025-10521-x, 2025.
- [20] Uzun, Y., Suraworachet, W., Zhou, Q., “Engagement with Analytics Feedback and Its Relationship to Self-Regulated Learning Competence and Course Performance,” *International Journal of Educational Technology in Higher Education*, doi:10.1186/s41239-025-00515-3, 2025.
- [21] Butt, A. U. R., Ali, H., Asif, M., “Enhancing Student Management Through Hybrid Machine Learning and Rough Set Models,” *IEEE Access*, doi:10.1109/access.2025.3565585, 2025.
- [22] “CLST: Cold-Start Mitigation in Knowledge Tracing by Aligning a Generative Language Model as a Students’ Knowledge Tracer,” arXiv:2406.10296, 2024.
- [23] “MAML-KT: Addressing the Cold-Start Problem in Knowledge Tracing for New Students via Few-Shot Model-Agnostic Meta-Learning,” arXiv:2603.00137, 2026.
- [24] “Automatic Item Generation in Various STEM Subjects Using Large Language Model Prompting,” *Computers and Education: Artificial Intelligence*, doi:10.1016/j.caeai.2024.100344, 2024.
- [25] “Self-hosted Lecture-to-Quiz: Local LLM MCQ Generation with Deterministic Quality Control,” arXiv:2603.08729, 2026.
- [26] “Automated Generation and Tagging of Knowledge Components from Multiple-Choice Questions,” doi:10.1145/3657604.3662030, 2024.
- [27] “Automatic Grading of Short Answers Using Large Language Models in Software Engineering Courses,” *Proc. EDUCON*, IEEE, doi:10.1109/educon60312.2024.10578839, 2024.
- [28] “Understanding Gaming the System by Analysing Self-Regulated Learning in Think-Aloud Protocols,” doi:10.1145/3785022.3785025, 2026.

- [29] “Personalised Forgetting Mechanism with Concept-Driven Knowledge Tracing,” doi:10.1145/3810940, 2026.
- [30] “Personalised Language Learning Using Spaced Repetition Scheduling,” doi:10.1007/978-3-031-98459-4_19, 2025.